

ЛАБОРАТОРНАЯ РАБОТА №16 (2+2+1+2+3 БАЛЛОВ)

ОДНОСВЯЗНЫЕ СПИСКИ

Вариант	Задача 1	Задача 2	Задача 3	Задача 4	Задача 5
1	Dynamic3	Dynamic24	ListWork1	ListWork22	ListWork59
2	Dynamic4	Dynamic23	ListWork2	ListWork23	ListWork60
3	Dynamic5	Dynamic22	ListWork3	ListWork24	ListWork61
4	Dynamic6	Dynamic21	ListWork4	ListWork25	ListWork62
5	Dynamic7	Dynamic20	ListWork5	ListWork22	ListWork59
6	Dynamic8	Dynamic19	ListWork6	ListWork23	ListWork60
7	Dynamic9	Dynamic18	ListWork7	ListWork24	ListWork61
8	Dynamic10	Dynamic17	ListWork8	ListWork25	ListWork62
9	Dynamic11	Dynamic16	ListWork9	ListWork22	ListWork59
10	Dynamic12	Dynamic15	ListWork10	ListWork23	ListWork60
11	Dynamic13	Dynamic14	ListWork11	ListWork24	ListWork61
12	Dynamic3	Dynamic25	ListWork12	ListWork25	ListWork62
13	Dynamic4	Dynamic26	ListWork13	ListWork22	ListWork60
14	Dynamic5	Dynamic27	ListWork14	ListWork23	ListWork61
15	Dynamic6	Dynamic28	ListWork15	ListWork24	ListWork62

**ВСЕ ДИНАМИЧЕСКИЕ СТРУКТУРЫ В ДАННОЙ ЛАБОРАТОРНОЙ РАБОТЕ
НУЖНО РЕАЛИЗОВАТЬ САМОСТОЯТЕЛЬНО С ПОМОЩЬЮ КЛАССОВ**

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ: СТЕК

В заданиях данной подгруппы структура «стек» (stack) моделируется цепочкой связанных узлов-объектов типа Node. Свойство Next последнего элемента цепочки равно null. *Вершиной стека* (top) считается первый элемент цепочки. Для доступа к стеку используется переменная типа Node — ссылка на вершину стека (для пустого стека данная переменная полагается равной null). *Значением* элемента стека считается значение его свойства Data.

Dynamic3. Дано число D и вершина A1 непустого стека. Добавить элемент со значением D в стек и вывести ссылку A2 на новую вершину стека.

Dynamic4. Дано число N (> 0) и набор из N чисел. Создать стек, содержащий исходные числа (последнее число будет вершиной стека), и вывести ссылку на его вершину.

Dynamic5. Дана вершина A1 непустого стека. Извлечь из стека первый (верхний) элемент и вывести его значение D, а также ссылку A2 на новую вершину стека. Если после извлечения элемента стек окажется пустым, то положить $A2 = \text{null}$. После извлечения элемента из стека освободить ресурсы, используемые этим элементом, вызвав его метод Dispose.

Dynamic6. Дана вершина A1 стека, содержащего не менее десяти элементов. Извлечь из стека первые девять элементов и вывести их значения. Вывести также ссылку на новую вершину стека. После извлечения элементов из стека освобождать ресурсы, которые они использовали, вызывая для этих элементов метод Dispose.

Dynamic7. Дана вершина A1 стека (если стек пуст, то $A1 = \text{null}$). Извлечь из стека все элементы и вывести их значения. Вывести также количество извлеченных элементов N (для пустого стека вывести 0). После извлечения элементов из стека освобождать ресурсы, которые они использовали, вызывая для этих элементов метод Dispose.

Dynamic8. Даны вершины A1 и A2 двух непустых стеков. Переместить все элементы из первого стека во второй (в результате элементы первого стека будут располагаться во втором стеке в порядке, обратном исходному) и вывести ссылку на новую вершину второго стека. Новые объекты типа Node не создавать.

Dynamic9. Даны вершины A1 и A2 двух непустых стеков. Перемещать элементы из первого стека во второй, пока значение вершины первого стека не станет четным (перемещенные элементы первого стека будут располагаться во втором стеке в порядке, обратном исходному). Если в первом стеке нет элементов с четными значениями, то переместить из первого стека во второй все элементы. Вывести ссылки на новые вершины первого и второго стека (если первый стек окажется пустым, то вывести для него константу null). Новые объекты типа Node не создавать.

Dynamic10. Дана вершина A1 непустого стека. Создать два новых стека, переместив в первый из них все элементы исходного стека с четными значениями, а во второй — с нечетными (элементы в новых стеках будут располагаться в порядке, обратном исходному; один из этих стеков может оказаться пустым). Вывести ссылки на вершины полученных стеков (для пустого стека вывести константу null). Новые объекты типа Node не создавать.

Dynamic11. Дана вершина A1 стека (если стек пуст, то A1 = null). Также дано число N (> 0) и набор из N чисел. Описать класс IntStack, содержащий следующие члены:

- закрытое поле top типа Node (вершина стека);
- конструктор с параметром aTop — вершиной существующего стека;
- процедура Push(D), которая добавляет в стек новый элемент со значением D (D — входной параметр целого типа);
- процедура Put (без параметров), которая выводит ссылку на поле top, используя метод Put класса PT.

С помощью метода Push добавить в исходный стек данный набор чисел (последнее число будет вершиной стека) и вывести ссылку на новую вершину стека, используя для этого метод Put класса IntStack.

Dynamic12. Дана вершина A1 стека, содержащего не менее пяти элементов. Включить в класс IntStack (см. задание Dynamic11) функцию Pop целого типа (без параметров), которая извлекает из стека первый (верхний) элемент, возвращает его значение и вызывает для него метод Dispose. С помощью метода Pop извлечь из исходного стека пять элементов и вывести их значения. Вывести также ссылку на новую вершину стека (если результирующий стек окажется пустым, то эта ссылка должна быть равна null).

Dynamic13. Дана вершина A1 стека. Включить в класс IntStack (см. задание Dynamic11) две функции: IsEmpty логического типа (возвращает true, если стек пуст, и false в противном случае) и Peek целого типа (возвращает значение вершины непустого стека, не удаляя ее из стека). Функции не имеют параметров. С помощью этих функций, а также функции Pop из задания Dynamic12, извлечь из исходного стека пять элементов (или все содержащиеся в нем элементы, если их менее пяти) и вывести их значения. Вывести также значение функции IsEmpty для результирующего стека и, если результирующий стек не является пустым, значение его новой вершины и ссылку на нее.

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ: ОЧЕРЕДЬ

В заданиях данной подгруппы структура «очередь» (queue) моделируется цепочкой связанных узлов-объектов типа Node. Свойство Next последнего элемента цепочки равно null. Началом очереди («головой», head) считается первый элемент цепочки, концом («хвостом», tail) — ее последний элемент. Для возможности быстрого добавления в конец очереди нового элемента удобно хранить, помимо ссылки на начало очереди, также и ссылку на ее конец. В случае пустой очереди ссылки на ее начало и конец полагаются равными null. Как и для стека, значением элемента очереди считается значение его свойства Data.

Dynamic14. Дан набор из 10 чисел. Создать очередь, содержащую данные числа в указанном порядке (первое число будет размещаться в начале очереди, последнее — в конце), и вывести ссылки A1 и A2 на начало и конец очереди.

Dynamic15. Дан набор из 10 чисел. Создать две очереди: первая должна содержать числа из исходного набора с нечетными номерами (1, 3, ..., 9), а вторая — с четными (2, 4, ..., 10);

порядок чисел в каждой очереди должен совпадать с порядком чисел в исходном наборе. Вывести ссылки на начало и конец первой, а затем второй очереди.

Dynamic16. Дан набор из 10 чисел. Создать две очереди: первая должна содержать все нечетные, а вторая — все четные числа из исходного набора (порядок чисел в каждой очереди должен совпадать с порядком чисел в исходном наборе). Вывести ссылки на начало и конец первой, а затем второй очереди (одна из очередей может оказаться пустой; в этом случае вывести для нее две константы null).

Dynamic17. Дано число D и ссылки $A1$ и $A2$ на начало и конец очереди (если очередь является пустой, то $A1 = A2 = \text{null}$). Добавить элемент со значением D в конец очереди и вывести ссылки на начало и конец полученной очереди.

Dynamic18. Дано число D и ссылки $A1$ и $A2$ на начало и конец очереди, содержащей не менее двух элементов. Добавить элемент со значением D в конец очереди и извлечь из очереди первый (начальный) элемент. Вывести значение извлеченного элемента, а также ссылки на начало и конец полученной очереди. После извлечения элемента из очереди вызвать для него метод `Dispose`.

Dynamic19. Дано число N (> 0) и ссылки $A1$ и $A2$ на начало и конец непустой очереди. Извлечь из очереди N начальных элементов и вывести их значения (если очередь содержит менее N элементов, то извлечь все ее элементы). Вывести также ссылки на начало и конец полученной очереди (для пустой очереди дважды вывести null). После извлечения элементов вызывать для них метод `Dispose`.

Dynamic20. Даны ссылки $A1$ и $A2$ на начало и конец непустой очереди. Извлекать из очереди элементы, пока значение начального элемента очереди не станет четным, и выводить значения извлеченных элементов (если очередь не содержит элементов с четными значениями, то извлечь все ее элементы). Вывести также ссылки на начало и конец полученной очереди (для пустой очереди дважды вывести null). После извлечения элементов вызывать для них метод `Dispose`.

Dynamic21. Даны две очереди; начало и конец первой равны $A1$ и $A2$, а второй — $A3$ и $A4$ (если очередь является пустой, то соответствующие объекты равны null). Переместить все элементы первой очереди (в порядке от начала к концу) в конец второй очереди и вывести ссылки на начало и конец преобразованной второй очереди. Новые объекты типа `Node` не создавать.

Dynamic22. Дано число N (> 0) и две непустые очереди; начало и конец первой равны $A1$ и $A2$, а второй — $A3$ и $A4$. Переместить N начальных элементов первой очереди в конец второй очереди. Если первая очередь содержит менее N элементов, то переместить из первой очереди во вторую все элементы. Вывести новые ссылки на начало и конец первой, а затем второй очереди (для пустой очереди дважды вывести null). Новые объекты типа `Node` не создавать.

Dynamic23. Даны две непустые очереди; начало и конец первой равны $A1$ и $A2$, а второй — $A3$ и $A4$. Перемещать элементы из начала первой очереди в конец второй, пока значение начального элемента первой очереди не станет четным (если первая очередь не содержит четных элементов, то переместить из первой очереди во вторую все элементы). Вывести новые ссылки на начало и конец первой, а затем второй очереди (для пустой очереди дважды вывести null). Новые объекты типа `Node` не создавать.

Dynamic24. Даны две непустые очереди; начало и конец первой равны $A1$ и $A2$, а второй — $A3$ и $A4$. Очереди содержат одинаковое количество элементов. Объединить очереди в одну, в которой элементы исходных очередей чередуются (начиная с первого элемента первой очереди). Вывести ссылки на начало и конец полученной очереди. Новые объекты типа `Node` не создавать.

Dynamic25. Даны две непустые очереди; начало и конец первой равны $A1$ и $A2$, а второй — $A3$ и $A4$. Элементы каждой из очередей упорядочены по возрастанию (в направлении от начала очереди к концу). Объединить очереди в одну с сохранением упорядоченности элементов. Вывести ссылки на начало и конец полученной очереди. Новые объекты типа `Node` не создавать, свойства `Data` не изменять.

Dynamic26. Даны ссылки A1 и A2 на начало и конец очереди (если очередь является пустой, то $A1 = A2 = \text{null}$). Также дано число $N (> 0)$ и набор из N чисел. Описать класс `IntQueue`, содержащий следующие члены:

- закрытые поля `head` и `tail` типа `Node` (начало и конец очереди);
- конструктор с параметрами `aHead`, `aTail` — началом и концом существующей очереди;
- процедура `Enqueue(D)`, которая добавляет в конец очереди новый элемент со значением D (D — входной параметр целого типа);
- процедура `Put` (без параметров), которая выводит ссылки на поля `head` и `tail`, используя метод `Put` класса `PT`.

С помощью метода `Enqueue` добавить в исходную очередь данный набор чисел и вывести новые ссылки на ее начало и конец, используя для этого метод `Put` класса `IntQueue`.

Dynamic27. Даны ссылки A1 и A2 на начало и конец очереди, содержащей не менее пяти элементов. Включить в класс `IntQueue` (см. задание `Dynamic26`) функцию `Dequeue` целого типа (без параметров), которая извлекает из очереди первый (начальный) элемент, возвращает его значение и вызывает для него метод `Dispose`. С помощью функции `Dequeue` извлечь из исходной очереди пять начальных элементов и вывести их значения. Вывести также ссылки на начало и конец результирующей очереди (если очередь окажется пустой, то эти ссылки должны быть равны `null`).

Dynamic28. Даны ссылки A1 и A2 на начало и конец очереди. Включить в класс `IntQueue` (см. задание `Dynamic26`) функцию `IsEmpty` логического типа (без параметров), которая возвращает `true`, если очередь пуста, и `false` в противном случае. Используя эту функцию для проверки состояния очереди, а также функцию `Dequeue` из задания `Dynamic27`, извлечь из исходной очереди пять начальных элементов (или все содержащиеся в ней элементы, если их менее пяти) и вывести их значения. Вывести также значение функции `IsEmpty` для полученной очереди и новые ссылки на ее начало и конец.

LISTWORK

ЛЕГКИЕ ЗАДАЧИ НА РАБОТУ С ОДНОСВЯЗНЫМИ ЛИНЕЙНЫМИ СПИСКАМИ

ListWork1. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на второй элемент этого списка P_2 . Известно, что в исходном списке не менее 2 элементов.

ListWork2. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на третий элемент этого списка P_3 . Известно, что в исходном списке не менее 3 элементов.

ListWork3. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на четвертый элемент этого списка P_4 . Известно, что в исходном списке не менее 5 элементов.

ListWork4. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на пятый элемент этого списка P_5 . Известно, что в исходном списке не менее 5 элементов.

ListWork5. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на шестой элемент этого списка P_6 . Известно, что в исходном списке не менее 6 элементов.

ListWork6. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на седьмой элемент этого списка P_7 . Известно, что в исходном списке не менее 7 элементов.

ListWork7. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на восьмой элемент этого списка P_8 . Известно, что в исходном списке не менее 8 элементов.

ListWork8. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на девятый элемент этого списка P_9 . Известно, что в исходном списке не менее 9 элементов.

ListWork9. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести указатель на последний элемент этого списка P_x .

ListWork10. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти первый элемент, кратный 4, и вывести указатель на этот элемент списка P_x . Известно, что такой элемент списка точно есть.

ListWork11. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти первый элемент, кратный 5, и вывести указатель на этот элемент списка P_x . Если такого элемента в списке нет, то результат должен быть равен nil.

ListWork12. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти второй элемент, кратный 6, и вывести указатель на этот элемент списка P_x . Если такого элемента в списке нет, то результат должен быть равен nil.

ListWork13. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо найти и вывести все элементы списка кратные трем. Известно, что хотя бы один такой элемент в списке точно есть.

ListWork14. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести элементы списка, перечисляя их от последнего к первому.

ListWork15. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вывести все элементы списка кратные 7 в порядке, обратном исходному. Известно, что в списке есть хотя бы два элемента кратные 7.

ЗАДАЧИ СРЕДНЕЙ СЛОЖНОСТИ НА РАБОТУ С ОДНОСВЯЗНЫМИ ЛИНЕЙНЫМИ СПИСКАМИ

ListWork22. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого второго элемента списка, и вывести ссылку на последний элемент полученного списка P_2 .

ListWork23. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого третьего элемента списка, и вывести ссылку на последний элемент полученного списка P_2 .

ListWork24. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого четвертого элемента списка, и вывести ссылку на последний элемент полученного списка P_2 .

ListWork25. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M перед каждым вторым элементом списка, и вывести ссылку на последний элемент полученного списка P_2 . При нечетном числе элементов исходного списка в конец списка вставлять не надо.

ListWork59. Дан односвязный линейный список и указатель на голову списка P_1 . Необходимо вставить значение M после каждого K -го элемента списка и вывести ссылку на последний элемент полученного списка P_2 .

ListWork60. Дан односвязный линейный список и указатель на голову списка P_1 . Значения элементов списка упорядочены по возрастанию. Необходимо создать копию исходного списка, после чего во вновь созданном списке вставить в список значение M так, чтобы он остался упорядоченным и вывести ссылку на первый элемент полученного списка P_2 .

ListWork61. Дан текстовый файл в первой строке которого хранится число N , а во второй строке N целых чисел. Необходимо создать упорядоченный по возрастанию список, в который поместить все эти элементы, при этом очередной элемент вставлять в список так, чтобы не нарушалась его упорядоченность.

ListWork62. Ден текстовый файл в первой строке которого хранится число N , а во второй строке N целых чисел. Необходимо создать упорядоченный по убыванию список, в который поместить все эти элементы, при этом очередной элемент вставлять в список так, чтобы не нарушалась его упорядоченность.